



QUESTÃO 1) (3,5 PONTOS) Faça uma função em linguagem C que receba uma matriz de caracteres, que contém apenas dígitos numéricos ('0' – '9'), e calcule a frequência de cada dígito numérico em cada coluna da matriz. O programa deve exibir o resultado em um formato de histograma para cada coluna, mostrando quantas vezes cada dígito aparece. A quantidade de linhas e colunas da matriz de caracteres estão definidas através da diretiva #define.

Protótipo: void histograma (^{CHAR}int mat[N][M])

Por exemplo, para uma matriz de 4x5:

$$M = \begin{bmatrix} '1' & '2' & '3' & '0' & '1' \\ '2' & '3' & '4' & '1' & '0' \\ '1' & '2' & '3' & '4' & '0' \\ '2' & '4' & '3' & '1' & '0' \end{bmatrix}$$

A saída correspondente seria:

Coluna 0:

0:
1: **
2: **
3:
4:
...

Coluna 1:

0:
1:
2: **
3: *
4: *
...

Coluna 2:

0:
1:
2:
3: ***
4: *
...

Coluna 3:

0: *
1: **
2:
3:
4: *
...

Coluna 4:

0: ***
1: *
2:
3:
4:
...

QUESTÃO 1) (3,5 PONTOS) Faça uma função em linguagem C que receba uma matriz de inteiros e ordene cada uma de suas linhas em ordem crescente. A quantidade de linhas e colunas da matriz estão definidas através da diretiva #define.

Protótipo: $\begin{matrix} & 3 & 4 \\ \text{void ordena_linhas} & (\text{int mat}[N][M]) \end{matrix}$

Por exemplo, para a matriz :

$$\text{mat} = \begin{bmatrix} 3 & 2 & 5 & 3 \\ 1 & 4 & 6 & 2 \\ 7 & 8 & 2 & 1 \end{bmatrix}$$

A matriz resultante seria:

$$\text{mat} = \begin{bmatrix} 2 & 3 & 3 & 5 \\ 1 & 2 & 4 & 6 \\ 1 & 2 & 7 & 8 \end{bmatrix}$$

QUESTÃO 2) (3,0 PONTOS) Escreva uma função em linguagem C que receba uma string representando um CPF no formato "XXX.XXX.XXX-YY", onde X e Y são dígitos verificadores. A função deve retornar 1 se os dígitos verificadores estiverem corretos e 0 se não estiverem.

Um CPF é composto por 11 dígitos, sendo os dois últimos dígitos são os dígitos verificadores. O cálculo do dígito verificador do CPF segue um algoritmo específico, que é baseado na fórmula conhecida como "Módulo 11". Começando da esquerda para a direita, multiplique cada dígito do CPF pela sequência de números 10, 9, 8, ..., 3, 2, nessa ordem. Por exemplo, utilizando o número: 123.456.789, é calculada a soma dos produtos dos nove dígitos utilizando:

$$(1 \times 10) + (2 \times 9) + (3 \times 8) + (4 \times 7) + (5 \times 6) + (6 \times 5) + (7 \times 4) + (8 \times 3) + (9 \times 2) = 210$$

O primeiro dígito verificador é 0 caso o resto da divisão por 11 da soma dos produtos seja 0 ou 1; caso contrário o dígito verificador corresponde a subtrair de 11 o resto da divisão por 11 da soma dos produtos. Por exemplo, o resto da divisão de 210 por 11 é 1 então o primeiro dígito verificador é 0.

Após o processo é repetido incluindo o primeiro dígito verificador, obtendo o valor 1234567890, e utilizando os pesos 11, 10, 9, 8, ..., 3, 2. Ou seja, a soma dos produtos é obtida utilizando:

$$(1 \times 11) + (2 \times 10) + (3 \times 9) + (4 \times 8) + (5 \times 7) + (6 \times 6) + (7 \times 5) + (8 \times 4) + (9 \times 3) + (0 \times 2) = 255$$

O segundo dígito verificador é 0 caso o resto da divisão da soma dos produtos seja 0 ou 1; caso contrário o dígito corresponde a 11 menos o resto da divisão por 11 da soma dos produtos. Exemplo, o resto da divisão de 255 por 11 é 2 então segundo dígito verificador é $11 - 2 = 9$.

Q1: VOID HISTOGRAMA (CHAR MAT[N][M]){

INT i, J, INDEXFREQ, K;

INT FREQ[M][10] = {0};

FOR(i=0; i<N; i++){

FOR(J=0; J<M; J++){

INDEXFREQ = MAT[i][J] - '0';

FREQ[X][INDEXFREQ] += 1;

}

}

FOR(i=0; i<M; i++){

PRINTF("COLUNA %d:\n", i);

FOR(J=0; J<10; J++){

PRINTF("%d: ", J);

FOR(K=0; K<FREQ[i][J]; K++){

PRINTF("*");

}

PRINTF("\n");

}

}

}

95

20

Q2: void ORDENA-LINHAS (int MAT[N][M]) {

int i, j, AUX, k;

FOR (i=0; i<N; i++) {

FOR (j=0; j<M-1; j++) {

FOR (k=0; k<M-j-1; k++) {

if (MAT[i][k] > MAT[i][k+1]) {

AUX = MAT[i][k];

MAT[i][k] = MAT[i][k+1];

MAT[i][k+1] = AUX;

}

}

}

}

}

Q3: int verificaDigVer(char CPF[15]) {

int i, SOMA = 0, DIGV1, DIGV2, MULTIPLICADOR = 10;

for (i = 0; CPF[i] != '-'; i++) {

SOMA += (CPF[i] - '0') * (MULTIPLICADOR - i); *OK*

if (CPF[i+1] == '.') {

i++;

MULTIPLICADOR++;

}

}

if (SOMA % 11 == 0 || SOMA % 11 == 1) DIGV1 = 0;

else DIGV1 = 11 - (SOMA % 11);

SOMA = 0;

MULTIPLICADOR = 11;

for (i = 0; CPF[i] != '-'; i++) {

SOMA += (CPF[i] - '0') * (MULTIPLICADOR - i);

if (CPF[i+1] == '.') {

i++;

MULTIPLICADOR++;

}

}

SOMA += DIGV1 * 2;

if (SOMA % 11 == 0 || SOMA % 11 == 1) DIGV2 = 0;

else DIGV2 = 11 - (SOMA % 11);

if ((DIGV1 == (CPF[12] - '0')) && (DIGV2 == (CPF[13] - '0')))

return 1;

else

return 0;

}